

Data Structure and Algorithm

CSEG2079

By: Dr. Mritunjay Shall Peelam
Assistant Professor Selection Grade,
Research Faculty and Senior Member IEEE
Website: <https://dr-mritunjaysp.com/>

Course Introduction and Objectives

	Data Structures	L	T	P	C
Version 1.0		3	0	0	3
Pre-requisites/Exposure	Basic Programming Knowledge				
Co-requisites	--				

Course Objectives

The objectives of this course are to:

1. Learn object oriented programing methodology.
2. Apply object oriented programming concepts to solve real life problem.
3. Implement Non Linear Data Structures by using C++ programming.

Course Introduction and Objectives

Course Outcomes

On completion of this course, the students will be able to

CO1.Demonstrate a clear understanding of the C++ programming environment and basic concepts.

CO2.Design solutions using the OOPs concepts.

CO3.Exhibit a clear understanding of Hash Tables, Binary trees and Binary Search trees.

CO4.Demonstrate the working of AVL trees, m-way search trees and Heap trees.

CO5.Develop skills to work with Graph data structures.

Catalog Description

This course introduces the basic concepts of object-oriented programming using C++. It provides deep insight into the theoretical concepts of designing and implementing data structures. Topics to be covered include introduction to C++ programming language, classes, pointers, functions, inheritance, polymorphism, file handling, tree (traversals & operations), binary tree, binary search tree, AVL tree, M-way tree, lists, trees, heaps, priority queues, sorting, hashing, graphs and graph traversals.

Syllabus

Course Content

UNIT I

8 Lecture Hours

C++ Class Overview- Class Definition, Objects, Class Members, Access Control, Constructors and destructors, parameter passing methods, Inline functions, static class members, this pointer, friend functions, dynamic memory allocation and de-allocation (new and delete).

UNIT II

12 Lecture Hours

Inheritance : Concept, base and derived classes, constructors and destructors, inheritance types, base class access control, Function over Loading, Operator Overloading, runtime polymorphism using virtual functions, abstract classes, Generic Programming- Function and class templates

UNIT III

8 Lecture Hours

Exception handling, File Handling, Hash Tables: Direct Address tables, hash functions: collision resolution-separate chaining, open addressing-linear probing, quadratic probing, double hashing, rehashing, extendible hashing.

UNIT IV

8 Lecture Hours

Trees- Binary tree, terminology, representation, complete binary tree, extend binary tree, traversing binary tree, threaded binary tree, Binary search tree & operations - Searching, Insertion and Deletion

UNIT V

6 Lecture Hours

AVL Trees, Operations – Insertion, Deletion and Searching, m-way search tree, operations-, insertion, deletion and searching, heaps, maintaining heaps property, heap sort, priority queue

UNIT VI

6 Lecture Hours

Graphs- terminology, multigraph, digraph, sequential representation, linked list representation, graph traversals (dfs&bfs), path matrix, completely connected graph.

Text Books and References

Text Books

1. Herbert Schildt, "C++: The Complete Reference", McGraw Hill Education, 2003.
2. John R. Hubbard, "Data Structures with C++", Schaum's Outlines, Tata McGraw Hill Education, 2000.

Reference Books

1. Michael T. Goodrich, Roberto Tamassia, David Mount, "Data Structures and Algorithms in C++", Wiley India Pvt. Ltd., 2004.
2. Seymour Lipschutz, "Data Structures", Schaum's Outlines, Tata McGraw Hill Education, 2006

Modes of Evaluation

Modes of Evaluation: Quiz/Assignment/ presentation/ extempore/ Written Examination
Examination Scheme:

Components	Internal	Mid Term	ESE
Weightage (%)	30%	20%	50%

Unit-1

Course Content

UNIT I

8 Lecture Hours

C++ Class Overview- Class Definition, Objects, Class Members, Access Control, Constructors and destructors, parameter passing methods, Inline functions, static class members, this pointer, friend functions, dynamic memory allocation and de-allocation (new and delete).

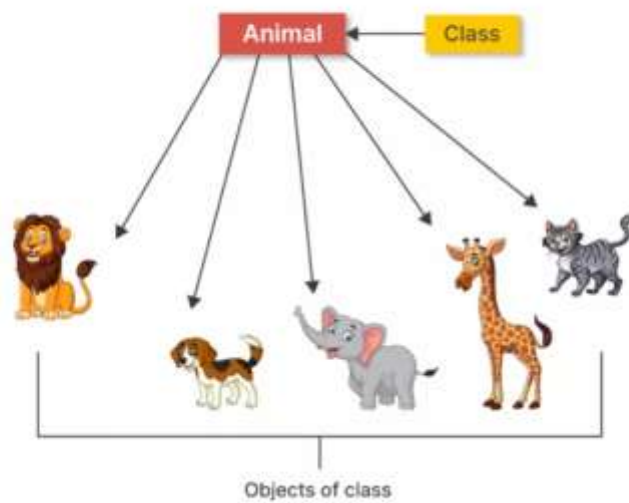
C++ Class Overview

- ☉ In the C++ programming language, classes and objects are the base building block that leads to object-oriented programming in C++.
- ☉ They provide the code in a more organized, reusable, and maintainable way.
- ☉ In C++, classes act like a blueprint that defines how objects are structured and behave in the program.
- ☉ On the other hand, objects are instances of classes that are used to hold data and methods to create and manipulate several entities.

C++ Class

- ☉ In C++, a class is a user-defined data type that acts as a blueprint to create an object.
- ☉ It is used to collect the data members and member functions into a single unit.
- ☉ A class keyword is used to create a class in C++.
- ☉ A class has several features, including fields, methods, constructors, etc.

C++ Class



All of them share common properties:

- a) 4 legs
- b) 1 tail
- c) Breeds
- d) Color

C++ Class

Syntax

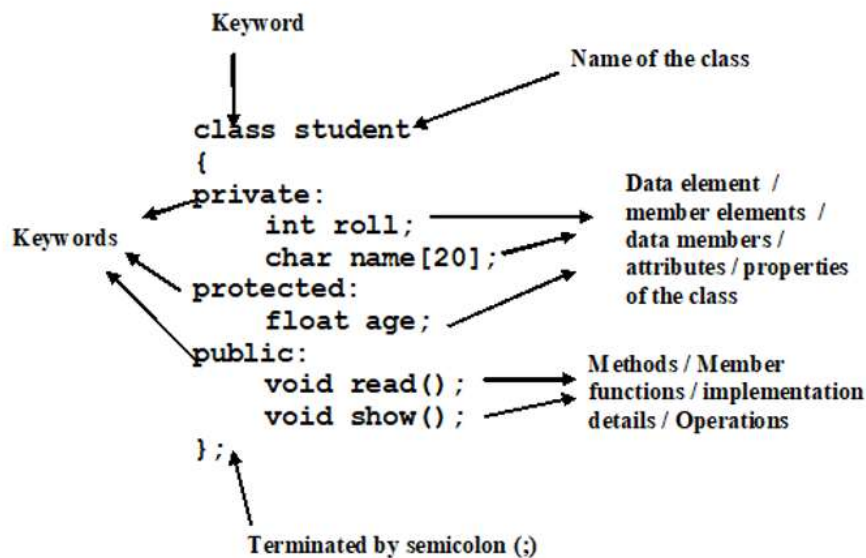
It has the following syntax:

```
class Class_Name {  
    public: // Access specifier  
  
    int data; // Data members  
    // Member functions  
    void display() {  
        // code to be executed  
    }  
};
```

C++ Class Members

- **Data Members:** Data members are the variables that are defined inside the class of the program.
- **Member Functions:** The functions that are declared inside a class are referred to as member method.
- **Access Specifiers:** These are the keywords (such as public, private, and protected) that are used to control the accessibility of data members and methods.

C++ Class Members



C++ Class Members

```
#include <iostream>
using namespace std; //using standard namespace

// Define a class named Employee
class Employee {
public:
    // Data members
    string name;
    int age;

    // Member function
    void displayInfo() {
        cout << "Enter the Name: " << name << endl;

        cout << "Enter the Age: " << age << endl;
    }
};

int main() //Main function
{
    // Create an object of Employee
    Employee e1;

    // Assign values to data members
    e1.name = "John";
    e1.age = 25;

    // Call member function
    e1.displayInfo();

    return 0;
}
```

Problems Output Debug Console Terminal Ports

```
PS C:\Users\mrity> cd "c:\Users\mrity\OneDrive\Documents\" ; if ($?) { gcc Hello.c -o Hello } ; if ($?) { .\Hello }
Hello, World!
PS C:\Users\mrity\OneDrive\Documents>
```

Access Control

- ✚ Access Control in C++ is a mechanism used to specify where the data members and member functions of a class can be accessed.
- ✚ It helps achieve data hiding, security, and encapsulation in Object-Oriented Programming.

C++ provides three access specifiers:

- 1. public**
- 2. private**
- 3. protected**

```
class ClassName
{
private:
    // Private data members
    // Private member functions

protected:
    // Protected data members
    // Protected member functions

public:
    // Public data members
    // Public member functions
};
```

Public Access Specifier

Members declared under public can be accessed:

- ✚ Inside the class
- ✚ Outside the class
- ✚ Through an object of the class
- ✚ Inside derived classes

```
class Student
{
public:
    int rollNo;
};
```

```
#include <iostream>
using namespace std;

class Student
{
public:
    string university;
    int rollNo;

    void display()
    {
        cout << "University: " << university << endl;
        cout << "Roll Number: " << rollNo << endl;
    }
};

int main()
{
    Student s1;

    // Public members can be accessed directly
    s1.university = "UPES";
    s1.rollNo = 101;

    s1.display();

    return 0;
}
```

Problems Output Debug Console Terminal Ports

```
PS C:\Users\mrityu\OneDrive\Documents> C:\Users\mrityu\OneDrive\Documents\hello.exe
University: UPES
Roll Number: 101
PS C:\Users\mrityu\OneDrive\Documents> █
```

Private Access Specifier

- ✚ Members declared as private can be accessed only inside the same class.
- ✚ They cannot be directly accessed through an object outside the class.

Syntax

```
class Student
{
private:
    int marks;
};
```

For a class, private is the default access specifier.

```
class Student
{
    int marks;
};
```

Example:

```
#include <iostream>
using namespace std;

class Student
{
private:
    int marks;

public:

    void setMarks(int m)
    {
        marks = m;
    }

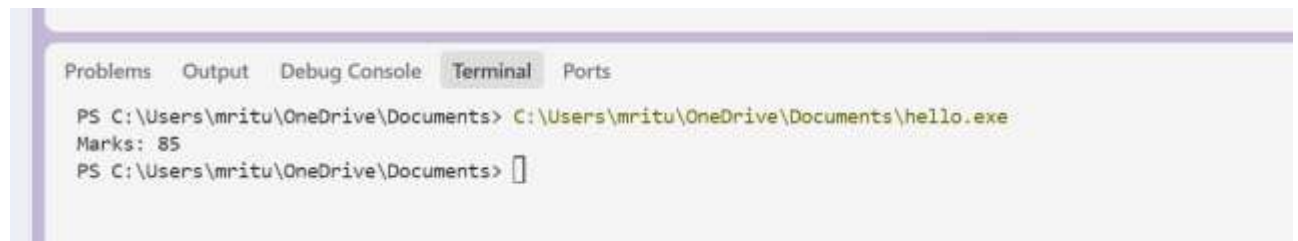
    void displayMarks()
    {
        cout << "Marks: " << marks << endl;
    }
};

int main()
{
    Student s1;
```

```
s1.setMarks(85);

s1.displayMarks();

return 0;
}
```



```
Problems Output Debug Console Terminal Ports
PS C:\Users\mrityu\OneDrive\Documents> C:\Users\mrityu\OneDrive\Documents\hello.exe
Marks: 85
PS C:\Users\mrityu\OneDrive\Documents> 
```

Protected Access Specifier

- ✚ Members declared as protected cannot normally be accessed directly outside the class.
- ✚ However, they can be accessed inside derived classes.
- ✚ Therefore, protected is mainly useful in inheritance.

Syntax

```
class Parent
{
protected:
    int value;
};
```

Example:

```
#include <iostream>
using namespace std;

class University
{
protected:
    string universityName;
};

class Student : public University
{
public:
```

```

void setUniversity()
{
    // Protected member can be accessed
    // inside the derived class
    universityName = "UPES";
}

void display()
{
    cout << "University: "
          << universityName << endl;
}
};

int main()
{
    Student s1;

    s1.setUniversity();
    s1.display();

    return 0;
}

```

```

Problems Output Debug Console Terminal Ports
PS C:\Users\mrity\OneDrive\Documents> C:\Users\mrity\OneDrive\Documents\hello.exe
University: UPES
PS C:\Users\mrity\OneDrive\Documents> 

```



Access Specifier	Same Class	Derived Class	Outside Through Object
public	Yes	Yes	Yes
private	Yes	No*	No
protected	Yes	Yes	No

***A derived class cannot directly access the private members of its base class.**

```
#include <iostream>
using namespace std;

class UPESStudent
{
private:
    int password;

protected:
    int internalMarks;

public:
    string name;

    void setData()
    {
        password = 1234;
        internalMarks = 40;
        name = "Aman";
    }

    void display()
    {
        cout << "Student Name: " << name << endl;
        cout << "Internal Marks: "
            << internalMarks << endl;
    }
};

int main()
{
    UPESStudent s1;

    s1.setData();

    // Public member
    cout << "Name: " << s1.name << endl;

    s1.display();
}
```

```
return 0;  
}
```



The screenshot shows a terminal window with tabs for Problems, Output, Debug Console, Terminal, and Ports. The Terminal tab is active, displaying the command prompt for a C++ program. The output shows the program's execution results.

```
PS C:\Users\mritu\OneDrive\Documents> C:\Users\mritu\OneDrive\Documents\hello.exe  
Name: Aman  
Student Name: Aman  
Internal Marks: 40  
PS C:\Users\mritu\OneDrive\Documents> 
```

Constructor in C++

- ✚ A constructor is a special member function of a class that is automatically called when an object of the class is created.
- ✚ It is primarily used to initialize the data members of an object.

Important Properties of a Constructor

- Constructor name is exactly the same as the class name.
- It does not have any return type, not even void.
- It is called automatically when an object is created.
- A constructor can have parameters.
- A class can contain multiple constructors.
- Constructors can be overloaded.
- Constructors are normally declared in the public section of a class.

Syntax:

```
class ClassName  
{  
public:  
  
    ClassName()  
    {  
        // Constructor body  
    }  
};
```

Example:

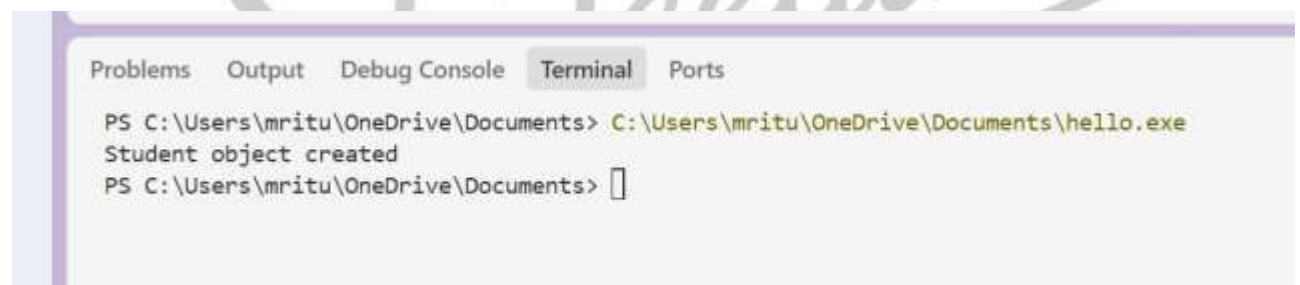
```
#include <iostream>
using namespace std;

class Student
{
public:

    Student()
    {
        cout << "Student object created" << endl;
    }
};

int main()
{
    Student s1;

    return 0;
}
```



The screenshot shows a terminal window with the following content:

```
Problems  Output  Debug Console  Terminal  Ports

PS C:\Users\mritu\OneDrive\Documents> C:\Users\mritu\OneDrive\Documents\hello.exe
Student object created
PS C:\Users\mritu\OneDrive\Documents> █
```

Types of Constructors

The major types of constructors in C++ are:

1. **Default Constructor**
2. **Parameterized Constructor**
3. **Copy Constructor**

Default Constructor

Definition

A **default constructor** is a constructor that can be called **without passing any arguments**.

A typical default constructor has no parameters.

Syntax

```
class Student
{
public:

    Student()
    {
        // Default constructor
    }
};
```

Example:

```
#include <iostream>
using namespace std;

class Student
{
private:
    string university;
    int rollNo;

public:

    // Default Constructor
    Student()
    {
        university = "UPES";
        rollNo = 101;
    }

    void display()
    {
        cout << "University: " << university << endl;
        cout << "Roll Number: " << rollNo << endl;
    }
};

int main()
{
    Student s1;

    s1.display();

    return 0;
}
```

```
Problems Output Debug Console Terminal Ports
PS C:\Users\mritu\OneDrive\Documents> C:\Users\mritu\OneDrive\Documents\hello.exe
University: UPES
Roll Number: 101
PS C:\Users\mritu\OneDrive\Documents> █
```

Parameterized Constructor

Definition

A **parameterized constructor** is a constructor that accepts one or more arguments during object creation.

It allows different objects to be initialized with different values.

Syntax

```
class ClassName
{
public:

    ClassName(parameter1, parameter2)
    {
        // Initialization
    }
};
```

Example:

```
#include <iostream>
using namespace std;

class Student
{
private:
    string name;
    int rollNo;

public:

    // Parameterized Constructor
    Student(string n, int r)
```

```

    {
        name = n;
        rollNo = r;
    }

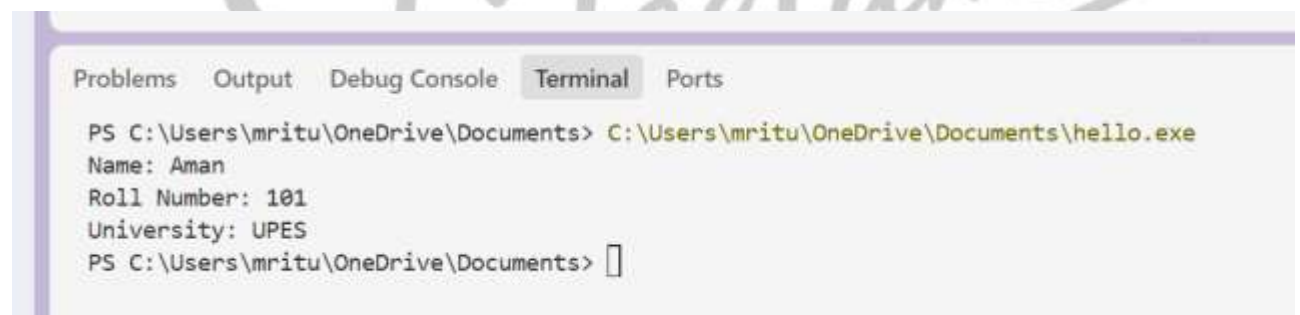
    void display()
    {
        cout << "Name: " << name << endl;
        cout << "Roll Number: " << rollNo << endl;
        cout << "University: UPES" << endl;
    }
};

int main()
{
    Student s1("Aman", 101);

    s1.display();

    return 0;
}

```



The screenshot shows a terminal window with tabs for Problems, Output, Debug Console, Terminal, and Ports. The Terminal tab is active, displaying the command prompt path and the execution of the program. The output matches the expected results from the code above.

```

PS C:\Users\mritu\OneDrive\Documents> C:\Users\mritu\OneDrive\Documents\hello.exe
Name: Aman
Roll Number: 101
University: UPES
PS C:\Users\mritu\OneDrive\Documents> 

```

```

#include <iostream>
using namespace std;

class Student
{
private:
    string name;
    int rollNo;

public:

    Student(string n, int r)
    {
        name = n;
        rollNo = r;
    }

    void display()

```

```

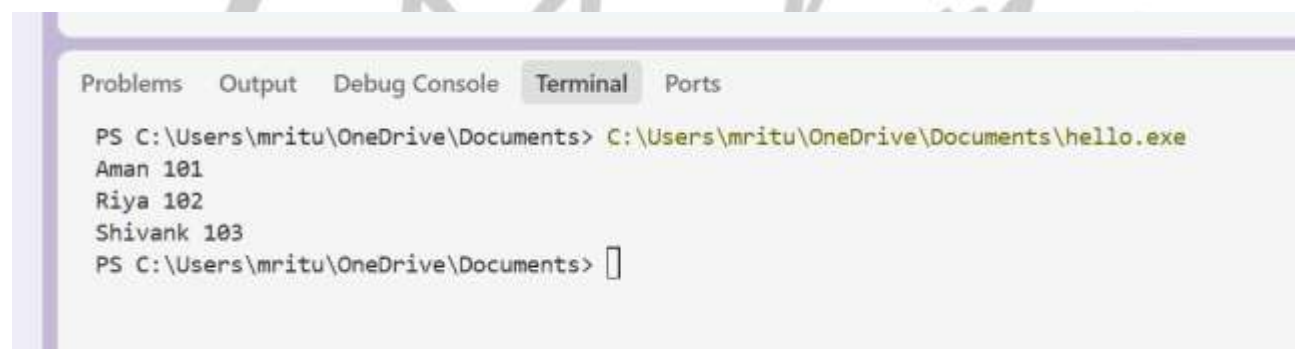
    {
        cout << name << " " << rollNo << endl;
    }
};

int main()
{
    Student s1("Aman", 101);
    Student s2("Riya", 102);
    Student s3("Shivank", 103);

    s1.display();
    s2.display();
    s3.display();

    return 0;
}

```



The screenshot shows a terminal window with tabs for Problems, Output, Debug Console, Terminal, and Ports. The Terminal tab is active, displaying the command prompt output of the program. The output shows three lines of student data: 'Aman 101', 'Riya 102', and 'Shivank 103', each on a new line. The prompt 'PS C:\Users\mritu\OneDrive\Documents>' is visible before and after the output.

```

PS C:\Users\mritu\OneDrive\Documents> C:\Users\mritu\OneDrive\Documents\hello.exe
Aman 101
Riya 102
Shivank 103
PS C:\Users\mritu\OneDrive\Documents> 

```

Copy Constructor

Definition

A **copy constructor** is a constructor that creates a new object by copying the data of an existing object of the same class.

Standard Syntax

```

ClassName(const ClassName &object)
{
    // Copy data
}

```

Example:

```

#include <iostream>
using namespace std;

```

```
class Student
{
private:
    string name;
    int rollNo;

public:

    Student(string n, int r)
    {
        name = n;
        rollNo = r;
    }

    // Copy Constructor
    Student(const Student &s)
    {
        name = s.name;
        rollNo = s.rollNo;

        cout << "Copy Constructor Called" << endl;
    }

    void display()
    {
        cout << "Name: " << name << endl;
        cout << "Roll Number: " << rollNo << endl;
    }
};

int main()
{
    Student s1("Aman", 101);

    Student s2 = s1;

    s2.display();

    return 0;
}
```

```
Problems Output Debug Console Terminal Ports
PS C:\Users\mritu\OneDrive\Documents> C:\Users\mritu\OneDrive\Documents\hello.exe
Copy Constructor Called
Name: Aman
Roll Number: 101
PS C:\Users\mritu\OneDrive\Documents> █
```

Constructor Overloading

Definition

Constructor overloading means defining more than one constructor in the same class with different parameter lists.

Example:

```
class Student
{
public:

    Student()
    {
        // Constructor 1
    }

    Student(int r)
    {
        // Constructor 2
    }

    Student(string n, int r)
    {
        // Constructor 3
    }
};
```

Example:

```
#include <iostream>
using namespace std;

class Student
{
private:
    string name;
```

```
    int rollNo;

public:

    // Default Constructor
    Student()
    {
        name = "Unknown";
        rollNo = 0;
    }

    // Constructor with one parameter
    Student(string n)
    {
        name = n;
        rollNo = 0;
    }

    // Constructor with two parameters
    Student(string n, int r)
    {
        name = n;
        rollNo = r;
    }

    void display()
    {
        cout << "Name: " << name
              << ", Roll No: " << rollNo << endl;
    }
};

int main()
{
    Student s1;
    Student s2("Aman");
    Student s3("Riya", 102);

    s1.display();
    s2.display();
    s3.display();

    return 0;
}
```

```
Problems Output Debug Console Terminal Ports
PS C:\Users\mritu\OneDrive\Documents> C:\Users\mritu\OneDrive\Documents\hello.exe
Name: Unknown, Roll No: 0
Name: Aman, Roll No: 0
Name: Riya, Roll No: 102
PS C:\Users\mritu\OneDrive\Documents> █
```

Constructor Initialization List

C++ provides another efficient method for initializing data members called the **member initializer list**.

Instead of:

```
Student(string n, int r)
{
    name = n;
    rollNo = r;
}
```

```
Student(string n, int r) : name(n), rollNo(r)
{
}
```

```
#include <iostream>
using namespace std;

class Student
{
private:
    string name;
    int rollNo;

public:
    Student(string n, int r) : name(n), rollNo(r)
    {
        cout << "Constructor called" << endl;
    }

    void display()
    {
```

```

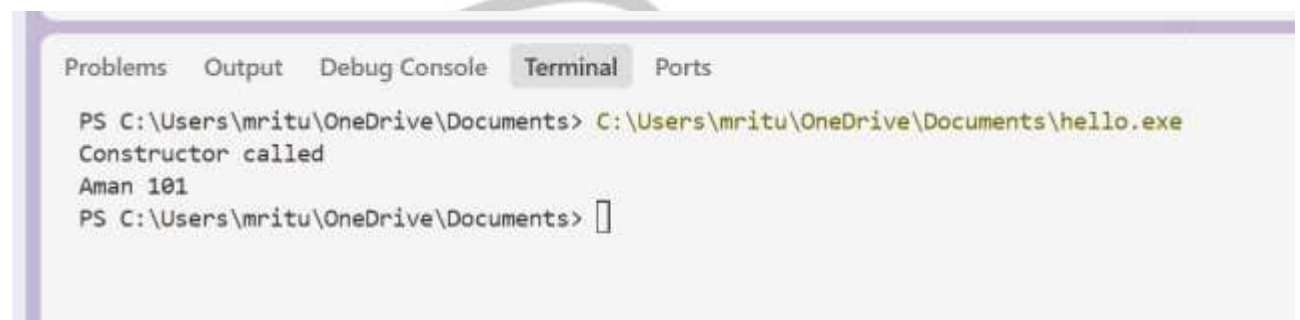
        cout << name << " " << rollNo << endl;
    }
};

int main()
{
    Student s1("Aman", 101);

    s1.display();

    return 0;
}

```



```

Problems  Output  Debug Console  Terminal  Ports

PS C:\Users\mritu\OneDrive\Documents> C:\Users\mritu\OneDrive\Documents\hello.exe
Constructor called
Aman 101
PS C:\Users\mritu\OneDrive\Documents> 

```

Destructor in C++

A **destructor** is a special member function of a class that is **automatically called when an object is destroyed**.

It is mainly used to release resources acquired by the object, such as dynamically allocated memory, files, locks, or other resources.

Syntax of Destructor

A destructor has the same name as the class preceded by the tilde symbol ~.

```

class Student
{
public:

    ~Student()
    {
        // Cleanup code
    }
};

```

Important Properties of Destructor

- Destructor name is the same as the class name.
- It starts with the ~ symbol.
- It has no return type.
- It takes no parameters.
- It cannot be overloaded in the usual way.
- A class has only one destructor.
- It is automatically called when an object's lifetime ends.
- It is commonly used for resource cleanup.

```
#include <iostream>
using namespace std;

class Student
{
public:

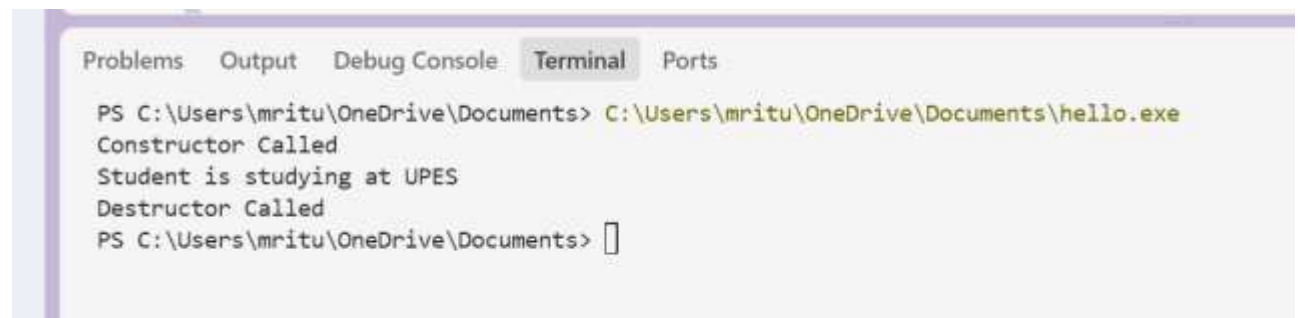
    Student()
    {
        cout << "Constructor Called" << endl;
    }

    ~Student()
    {
        cout << "Destructor Called" << endl;
    }
};

int main()
{
    Student s1;

    cout << "Student is studying at UPES" << endl;

    return 0;
}
```

A screenshot of a code editor's terminal window. The terminal shows the output of a C++ program. The first line is the command prompt 'PS C:\Users\mritu\OneDrive\Documents>' followed by the command 'C:\Users\mritu\OneDrive\Documents\hello.exe'. The output consists of three lines: 'Constructor Called', 'Student is studying at UPES', and 'Destructor Called'. The prompt is shown again at the end, indicating the program has finished execution.

```
Problems  Output  Debug Console  Terminal  Ports

PS C:\Users\mritu\OneDrive\Documents> C:\Users\mritu\OneDrive\Documents\hello.exe
Constructor Called
Student is studying at UPES
Destructor Called
PS C:\Users\mritu\OneDrive\Documents> 
```

The constructor represents the **beginning of the object's lifetime**, while the destructor represents its **cleanup at the end of that lifetime**.

```
#include <iostream>
using namespace std;

class Student
{
private:
    int id;

public:

    Student(int x)
    {
        id = x;

        cout << "Constructor for Student "
              << id << endl;
    }

    ~Student()
    {
        cout << "Destructor for Student "
              << id << endl;
    }
};

int main()
{
    Student s1(101);
    Student s2(102);
    Student s3(103);

    return 0;
}
```

Problems Output Debug Console Terminal Ports

```
PS C:\Users\mrityu\OneDrive\Documents> C:\Users\mrityu\OneDrive\Documents\hello.exe
Constructor for Student 101
Constructor for Student 102
Constructor for Student 103
Destructor for Student 103
Destructor for Student 102
Destructor for Student 101
PS C:\Users\mrityu\OneDrive\Documents> █
```

Local objects created in the same scope are normally destroyed in the reverse order of construction.

Feature	Constructor	Destructor
Purpose	Initialize an object	Clean up an object
Name	Same as class	Same as class with ~
Example	Student()	~Student()
Return Type	None	None
Parameters	Can have parameters	Cannot have parameters
Overloading	Possible	Not overloaded
Called	Object creation	Object destruction
Main Role	Initialization/resource acquisition	Cleanup/resource release

Parameter passing methods:

Parameter passing is the process of transferring data from a calling function to another function through its parameters.

For example:

```
sum(10, 20);
```

In C++, the main parameter passing methods are:

- ✚ Call by Value
- ✚ Call by Reference
- ✚ Call by Address / Pointer
- ✚ Passing by Constant Reference

1. Call by Value

- ✚ In Call by Value, a copy of the actual argument is passed to the function.
- ✚ Therefore, any modification made inside the function does not affect the original variable.

Syntax

```
void functionName(int x)
```

```
{
```

```
    // x contains a copy of the original value
```

```
}
```

```
#include <iostream>
using namespace std;

// Function receives a COPY of marks
void updateMarks(int marks)
{
    marks = marks + 5;    // Changes only the local copy

    cout << "Marks inside function = "
         << marks << endl;
}

int main()
{
    int marks = 80;

    cout << "Marks before function call = "
         << marks << endl;

    // Passing marks by value
    updateMarks(marks);

    cout << "Marks after function call = "
         << marks << endl;

    return 0;
}
```

Problems Output Debug Console **Terminal** Ports

```
PS C:\Users\mritu\OneDrive\Documents> C:\Users\mritu\OneDrive\Documents\demo.exe
Marks before function call = 80
Marks inside function = 85
Marks after function call = 80
PS C:\Users\mritu\OneDrive\Documents>
```

Call by Reference

- ✚ In Call by Reference, the function parameter becomes a reference or alias of the original variable.
- ✚ Therefore, modifications performed inside the function directly affect the original variable.

The reference operator & is used.

Syntax

```
void functionName(int &x)
{
    // x refers to the original variable
}
```

```
#include <iostream>
using namespace std;

// marks is a reference to the original variable
void updateMarks(int &marks)
{
    marks = marks + 5;    // Changes original variable

    cout << "Marks inside function = "
         << marks << endl;
}

int main()
```

```

{
    int marks = 80;

    cout << "Marks before function call = "
          << marks << endl;

    // Passing marks by reference
    updateMarks(marks);

    cout << "Marks after function call = "
          << marks << endl;

    return 0;
}

```

```

Problems  Output  Debug Console  Terminal  Ports

PS C:\Users\mritu\OneDrive\Documents> C:\Users\mritu\OneDrive\Documents\demo.exe
Marks before function call = 80
Marks inside function = 85
Marks after function call = 85
PS C:\Users\mritu\OneDrive\Documents> 

```

Call by Address / Pointer

- In Call by Address, the memory address of the variable is passed to the function.
- The function receives the address using a pointer.
- The address operator & and dereference operator * are used.

Syntax:

```

void functionName(int *x)
{
    *x = value;
}

```

Function call:

functionName(&variable);

```

#include <iostream>
using namespace std;

// Function receives address of CGPA
void updateCGPA(float *cgpa)
{
    // *cgpa accesses the value stored
    // at the given memory address
    *cgpa = *cgpa + 0.5;

    cout << "CGPA inside function = "
         << *cgpa << endl;
}

int main()
{
    float cgpa = 8.0;

    cout << "CGPA before function call = "
         << cgpa << endl;

    // Passing address of cgpa
    updateCGPA(&cgpa);

    cout << "CGPA after function call = "
         << cgpa << endl;

    return 0;
}

```



```

Problems  Output  Debug Console  Terminal  Ports

PS C:\Users\mritu\OneDrive\Documents> C:\Users\mritu\OneDrive\Documents\demo.exe
CGPA before function call = 8
CGPA inside function = 8.5
CGPA after function call = 8.5
PS C:\Users\mritu\OneDrive\Documents> 

```

Difference Between & and *

These two symbols are very important when working with pointers.

& Address Operator

It returns the memory address of a variable.

Example:

```
int x = 10;
```

```
cout << &x;
```

It may display something like:

```
0x61ff08
```

* Dereference Operator

It accesses the value stored at a particular memory address.

Example:

```
int x = 10;
```

```
int *ptr = &x;
```

```
cout << *ptr;
```

Output:

```
10
```

4. Passing by Constant Reference

✚ Sometimes we want to avoid creating a copy of a large object but also do not want the function to modify it.

✚ In such situations, we use a **constant reference**.

Syntax

```
void functionName(const string &x)
{
}
```

```

#include <iostream>
#include <string>
using namespace std;

// name is passed by reference
// but cannot be modified
void displayStudent(const string &name)
{
    cout << "Student Name: "
          << name << endl;

    // name = "Rahul";
    // Error: const reference cannot be modified
}

int main()
{
    string studentName = "Aman";

    displayStudent(studentName);

    return 0;
}

```

Problems Output Debug Console Terminal Ports

```

PS C:\Users\mritu\OneDrive\Documents> C:\Users\mritu\OneDrive\Documents\demo.exe
Student Name: Aman
PS C:\Users\mritu\OneDrive\Documents> 

```

Example: Swapping Two Numbers

This is one of the most common examples used to understand parameter passing.

Swapping Using Call by Value

```

#include <iostream>
using namespace std;

// Call by Value
void swapNumbers(int a, int b)
{
    int temp;

    temp = a;
    a = b;
    b = temp;
}

```

```

        cout << "Inside function:" << endl;
        cout << "a = " << a << endl;
        cout << "b = " << b << endl;
    }

    int main()
    {
        int a = 10;
        int b = 20;

        cout << "Before swapping:" << endl;
        cout << "a = " << a << endl;
        cout << "b = " << b << endl;

        swapNumbers(a, b);

        cout << "\nAfter function call:" << endl;
        cout << "a = " << a << endl;
        cout << "b = " << b << endl;

        return 0;
    }

```

Output

Before swapping:

a = 10
b = 20

Inside function:

a = 20
b = 10

After function call:

a = 10
b = 20

The original values are not swapped because only copies were modified.

Swapping Using Call by Reference

```

#include <iostream>
using namespace std;

// Call by Reference
void swapNumbers(int &a, int &b)
{
    int temp;

    temp = a;
    a = b;
    b = temp;
}

```

```

}

int main()
{
    int a = 10;
    int b = 20;

    cout << "Before swapping:" << endl;

    cout << "a = " << a << endl;
    cout << "b = " << b << endl;

    // Passing variables by reference
    swapNumbers(a, b);

    cout << "\nAfter swapping:" << endl;

    cout << "a = " << a << endl;
    cout << "b = " << b << endl;

    return 0;
}

```

Output

Before swapping:

a = 10
b = 20

After swapping:

a = 20
b = 10

Here the original variables are modified.

Swapping Using Pointer

```

#include <iostream>
using namespace std;

// Function receives addresses
void swapNumbers(int *a, int *b)
{
    int temp;

    // Access value using dereference operator
    temp = *a;

    *a = *b;

    *b = temp;
}

```

```

int main()
{
    int a = 10;
    int b = 20;

    cout << "Before swapping:" << endl;

    cout << "a = " << a << endl;
    cout << "b = " << b << endl;

    // Passing addresses
    swapNumbers(&a, &b);

    cout << "\nAfter swapping:" << endl;

    cout << "a = " << a << endl;
    cout << "b = " << b << endl;

    return 0;
}

```

Output

Before swapping:

a = 10

b = 20

After swapping:

a = 20

b = 10

Feature	Call by Value	Call by Reference	Call by Address
What is passed?	Copy of value	Reference to variable	Address of variable
Syntax	int x	int &x	int *x
Function call	fun (a)	fun (a)	fun (&a)
Original data modified?	No	Yes	Yes
Uses pointer?	No	No	Yes
Uses & in parameter?	No	Yes	No
Dereference required?	No	No	Yes
Easy to use	Very easy	Easy	Comparatively complex

Inline Functions in C++

- ✚ An **inline function** is a function for which the compiler is requested to insert the function code directly at the place where the function is called.
- ✚ Normally, when a function is called, the program transfers control to the function and returns after execution. In an inline function, this function-call overhead may be reduced because the compiler can replace the function call with the actual function body.

The keyword used is:

`inline`

Syntax

```
inline returnType functionName(parameters)
{
    // Function body
}
```

```
#include <iostream>
using namespace std;

// Inline function to calculate square
inline int square(int x)
{
    return x * x;
}

int main()
{
    int number = 5;

    // Calling inline function
    cout << "Square = " << square(number);

    return 0;
}
```

Problems Output Debug Console Terminal Ports

```
PS C:\Users\mrityu\OneDrive\Documents> C:\Users\mrityu\OneDrive\Documents\demo.exe
Student Name: Aman
PS C:\Users\mrityu\OneDrive\Documents> C:\Users\mrityu\OneDrive\Documents\demo.exe
Square = 25
PS C:\Users\mrityu\OneDrive\Documents> █
```

Important Points

- ✚ Inline functions are suitable for **small and frequently called functions**.
- ✚ They may improve performance by reducing function-call overhead.
- ✚ `inline` is only a **request to the compiler**.
- ✚ The compiler may decide not to inline a function.
- ✚ Large functions should generally not be made inline.
- ✚ Recursive functions may not be suitable for inlining.

Static Class Members

- ✚ A **static class member** belongs to the **class itself rather than to an individual object**.
- ✚ This means all objects of a class share the same static data member.
- ✚ Only **one copy** of a static data member exists, irrespective of how many objects are created.

Syntax

Inside the class:

```
class ClassName
{
    static int variable;
};
```

Definition outside the class:

```
int ClassName::variable = 0;
```

```
#include <iostream>
using namespace std;

class Student
{
private:
    // Shared by all objects
    static int count;

public:
    Student()
    {
        // Increment whenever an object is created
        count++;
    }
};
```

```

    void showCount()
    {
        cout << "Total Students = " << count << endl;
    }
};

// Definition of static data member
int Student::count = 0;

int main()
{
    Student s1;
    Student s2;
    Student s3;

    s1.showCount();

    return 0;
}

```

```

Problems  Output  Debug Console  Terminal  Ports
PS C:\Users\mrityu\OneDrive\Documents> C:\Users\mrityu\OneDrive\Documents\demo.exe
Total Students = 3
PS C:\Users\mrityu\OneDrive\Documents> 

```

Static Member Function

A **static member function** belongs to the class instead of an individual object.

It can be called using:

```
ClassName::functionName();
```

Example:

```
UPESStudent::displayTotal();
```

A static member function can directly access **static members** of the class.

this Pointer in C++

- ✚ The **this** pointer is a special pointer automatically available inside every **non-static member function** of a class.
- ✚ It points to the **current object that called the member function**.

In simple words:

`this = Address of current object`

Syntax

`this->variable`

Why Is `this` Pointer Required?

It is especially useful when the **parameter name and class data member name are the same**.

```
#include <iostream>
using namespace std;

class Student
{
private:
    int marks;

public:

    void setMarks(int marks)
    {
        // this->marks refers to class data member
        // marks refers to function parameter
        this->marks = marks;
    }

    void display()
    {
        cout << "Marks = " << this->marks << endl;
    }
};

int main()
{
    Student s1;
```

```
s1.setMarks(90);

s1.display();

return 0;
}
```

```
Problems  Output  Debug Console  Terminal  Ports

PS C:\Users\mritu\OneDrive\Documents> C:\Users\mritu\OneDrive\Documents\demo.exe
Marks = 90
PS C:\Users\mritu\OneDrive\Documents> █
```

Friend Functions

A **friend function** is a function that is **not a member of a class**, but it is allowed to access the **private and protected members** of that class.

The keyword used is:

friend

```
#include <iostream>
using namespace std;

class Student
{
private:
    int marks;

public:

    Student()
    {
        marks = 90;
    }

    // Friend function declaration
    friend void displayMarks(Student s);
};
```

```
// Friend function definition
void displayMarks(Student s)
{
    // Accessing private member directly
    cout << "Marks = " << s.marks << endl;
}

int main()
{
    Student s1;

    // Calling friend function
    displayMarks(s1);

    return 0;
}
```

```
Problems Output Debug Console Terminal Ports
PS C:\Users\mrityu\OneDrive\Documents> C:\Users\mrityu\OneDrive\Documents\demo.exe
Marks = 90
PS C:\Users\mrityu\OneDrive\Documents> █
```

Feature	Member Function	Friend Function
Part of class	Yes	No
Declared using <code>friend</code>	No	Yes
Can access private data	Yes	Yes
Called using object	Usually yes	No
Uses <code>this</code> pointer	Yes	No
Defined inside/outside class	Both possible	Usually outside class

Properties of Friend Function in C++

The important properties of a friend function are:

- ✚ It is not a member of the class
 - A friend function is an external function, even though it is declared inside the class.
- ✚ It can access private members
 - It can directly access the private data of the class.

✚ It can access protected members

A friend function can also access protected members of the class.

✚ It is declared using the friend keyword

Dynamic Memory Allocation and De-allocation

✚ Dynamic Memory Allocation (DMA) is the process of allocating memory during program execution (runtime) according to the program's requirements.

✚ In C++, dynamic memory is allocated from the heap memory using the **new** operator and released using the **delete** operator.

The two main operators are:

- new → allocates memory dynamically.
- delete → releases dynamically allocated memory.

Why Do We Need Dynamic Memory Allocation?

In normal variable declaration:

int marks;

✚ memory is allocated automatically and its size is fixed.

✚ However, sometimes we do not know in advance how much memory will be required.

For example, suppose a UPES program asks:

Enter number of students: 5

or:

Enter number of students: 500

The required memory depends on the input given at runtime. In such situations, dynamic memory allocation is useful.

new Operator

- ✚ The **new operator** dynamically allocates memory from the heap and returns the **address of the allocated memory**.
- ✚ Therefore, a pointer is generally used to store this address.

Syntax

```
data_type *pointer_name = new data_type;
```

Example

```
int *ptr = new int;
```

```
#include <iostream>
using namespace std;

int main()
{
    // Dynamically allocate memory for an integer
    int *ptr = new int;

    // Store value in dynamically allocated memory
    *ptr = 85;

    // Display value
    cout << "Marks = " << *ptr << endl;

    return 0;
}
```

Problems Output Debug Console **Terminal** Ports

```
PS C:\Users\mrityu\OneDrive\Documents> C:\Users\mrityu\OneDrive\Documents\demo.exe
Marks = 85
PS C:\Users\mrityu\OneDrive\Documents> █
```

delete Operator

The `delete` operator is used to release memory that was previously allocated using `new`.

Syntax

```
delete pointer_name;
```

Example

```
delete ptr;
```

After using `delete`, it is a good practice to assign:

```
ptr = nullptr;
```

This prevents the pointer from pointing to an invalid memory location.

```
#include <iostream>
using namespace std;

int main()
{
    // Dynamically allocate memory
    int *ptr = new int;

    // Assign value
    *ptr = 90;

    cout << "Value = " << *ptr << endl;

    // Release dynamically allocated memory
    delete ptr;

    // Avoid dangling pointer
    ptr = nullptr;

    return 0;
}
```

```
Problems Output Debug Console Terminal Ports
PS C:\Users\mrITU\OneDrive\Documents> C:\Users\mrITU\OneDrive\Documents\demo.exe
Value = 98
PS C:\Users\mrITU\OneDrive\Documents> █
```

Properties of Dynamic Memory Allocation

- ✚ Memory is allocated **at runtime**.
- ✚ Memory is generally allocated from the **heap**.
- ✚ `new` is used to allocate memory.
- ✚ `delete` is used to release memory.
- ✚ `new` returns the **address** of allocated memory.
- ✚ A pointer is used to store this address.
- ✚ The size of dynamic arrays can be determined at runtime.
- ✚ Dynamic memory remains allocated until it is explicitly released or the program ends.
- ✚ Improper deallocation can cause **memory leaks**.
- ✚ Accessing released memory can lead to **dangling-pointer errors**.

In C++, `new` and `delete` are generally preferred, but `malloc()`, `calloc()`, `realloc()`, and `free()` can also be used through the C standard library.

Dangling Pointer in C++

A **dangling pointer** is a pointer that refers to a memory location that has already been released or is no longer valid.

For example:

```
int *ptr = new int(50);

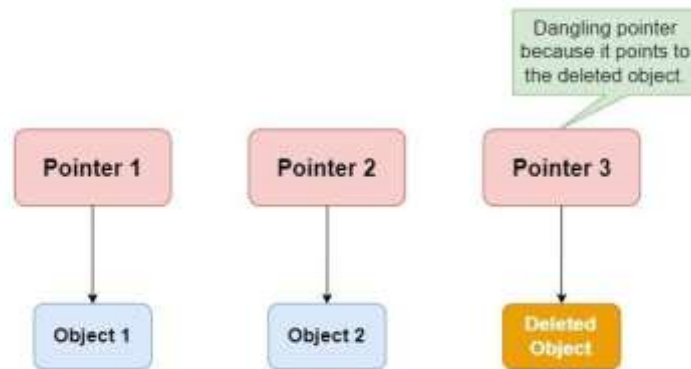
delete ptr;
```

After:

```
delete ptr;
```

The dynamically allocated memory is released, but `ptr` may still contain the old address.

Therefore, `ptr` becomes a **dangling pointer**.



```
#include <iostream>
using namespace std;

int main()
{
    int *ptr = new int;

    *ptr = 100;

    cout << "Value = " << *ptr << endl;

    // Release memory
    delete ptr;

    // ptr is now dangling

    return 0;
}
```

How to Avoid a Dangling Pointer

Set the pointer to `nullptr` after deleting the memory.

```
delete ptr;
ptr = nullptr;
```

```
#include <iostream>
using namespace std;

int main()
{
    int *ptr = new int(100);
```

```
cout << "Value = " << *ptr << endl;

delete ptr;

// Prevent dangling pointer
ptr = nullptr;

if (ptr == nullptr)
{
    cout << "Pointer is now nullptr." << endl;
}

return 0;
}
```

Meelam